

Week 1. Problem set (solutions by Karim Khabibrakhmanov)

1. Compute asymptotic worst case time complexity of the following algorithm (see pseudocode conventions in [CLRS, §2.1]). You **must** use Θ -notation. For justification, provide execution cost and frequency count for each line in the body of the **secret** procedure. Optionally, you may provide the details for the computation of the running time $T(n)$ for worst case scenario. Proof for the asymptotic bound is not required for this exercise.

	Line	Execution Cost	Frequency Count
1	/* A is a 0-indexed array,	—	—
2	* n is the number of items in A */	—	—
3	secret (A, n):	—	—
4	for i = 1 to n		
5	s := 1		
6	j := i - 1		
7	while j < n and A[j] ≥ A[i-1]		
8	j := j + s		
9	s := s + 2		
10	exchange A[i-1] with A[min(j, n - 1)]		

Answer: $\Theta(n^2)$.

Justification: cost and frequency count is presented in the following table:

Line	Execution Cost	Frequency Count
1	0	1
2	0	1
3	c_0	1
4	c_1	$n+1$
5	c_2	n
6	c_3	n
7	c_4	$\sum_{i=1}^n t_i$
8	c_5	$\sum_{i=1}^n (t_i - 1)$
9	c_6	$\sum_{i=1}^n (t_i - 1)$
10	c_7	n

(optional part) From this table, we conclude

$$T(n) = c_0 \cdot 1 + c_1 \cdot (n+1) + c_2 \cdot n + c_3 \cdot n + c_4 \cdot \sum_{i=1}^n t_i + c_5 \cdot \sum_{i=1}^n (t_i - 1) + c_6 \cdot \sum_{i=1}^n (t_i - 1) + c_7 \cdot n = \Theta(n^2)$$

2. Indicate, for each pair of expressions (A, B) in the table below whether $A = O(B)$, $A = o(B)$, $A = \Omega(B)$, $A = \omega(B)$, or $A = \Theta(B)$. Write your answer in the form of the table with YES or NO written in each cell:

A	B	$A = O(B)$	$A = o(B)$	$A = \Omega(B)$	$A = \omega(B)$	$A = \Theta(B)$
$1 + \sin(n)$	$(1 + \frac{1}{n})^{\frac{1}{n}}$	YES	NO	NO	NO	NO
$\sqrt[5]{n}$	$\log_3^2 n$	NO	NO	YES	YES	NO
n^2	$\log_2^n n$	YES	YES	NO	NO	NO
$(1 + \frac{1}{10^{100}})^n$	$n^{(10^{100})}$	NO	NO	YES	YES	NO
$\log_2((n+1)!)$	$n \log_2 n$	YES	NO	YES	NO	YES

3. Let f and g be functions from positive integers to positive reals. Assume $f(n) > n$ for $n > 2025$. Using the formal definition of asymptotic notation [CLRS, §3.2], prove *formally* that

$$\min(f(n) - \sqrt{n}, g(n) + n) = O(f(n) + g(n))$$

Proof. By definition of big-O notation, we need to show that there exist constant c and n_0 such that for all $n \geq n_0$

$$\min(f(n) - \sqrt{n}, g(n) + n) \leq c * (f(n) + g(n))$$

Let $c = 1$ and $n_0 = 2025$

By assumption, we have $f(n) > n$ for $n > 2025$

Consider the following cases:

- 1) if $(f(n) - \sqrt{n}) \geq (g(n) + n)$:

$$g(n) + n \leq 1 * (f(n) + g(n))$$

$$g(n) + n \leq 1 * f(n) + 1 * g(n)$$

Here we subtract $g(n)$ from both sides.

$$n \leq f(n)$$

Since we have established that $f(n) > n$ for $n > 2025$, the inequality holds true.

Thus, the condition $n \leq f(n)$ is satisfied for $n > 2025$.

- 2) if $(f(n) - \sqrt{n}) \leq (g(n) + n)$:

$$f(n) - \sqrt{n} \leq 1 * (f(n) + g(n))$$

$$f(n) - \sqrt{n} \leq 1 * f(n) + 1 * g(n)$$

Here we subtract $f(n)$ from both sides.

$$-\sqrt{n} \leq g(n)$$

Since $g(n)$ is a positive function and $n > 2025$, it follows that $\sqrt{n}$ is also positive. Therefore,

$-\sqrt{n}$ is negative. This implies that the inequality $-\sqrt{n} \leq g(n)$ holds true, as a negative value is always less than or equal to a positive value. Thus, the condition is satisfied.

- 3) Conclusion:

We have examined all possible cases for Big-O, and both have been found to be true.

Therefore, the statement $\min(f(n) - \sqrt{n}, g(n) + n) = O(f(n) + g(n))$ is also true.

QED.

□

References

- [CLRS] Cormen, T.H., Leiserson, C.E., Rivest, R.L. and Stein, C., 2022. *Introduction to algorithms, Fourth Edition*. MIT press.